MERN STACK

DAY – 36

5 August 2026

PROJECT: PARKEASY - PARKING MANAGEMENT SYSTEM

PARKING SLOT MANAGEMENT

• The Parking Slot Management module is developed to allow administrators to manage all parking slots in the ParkEasy system.
• It enables administrators to add new parking slots, update slot status, delete existing slots, and monitor slot availability.
• The module is implemented using React.js for the frontend and Express.js with MongoDB for backend operations.
• The useState() hook is used to store slot information, input values, and success messages.
• The useEffect() hook is used to automatically load parking slot data when the page is opened.
• The Sidebar component provides easy navigation within the administrator dashboard.
• The Parking Slot Management module communicates with the backend using multiple REST APIs.

Slot Management APIs

POST /slots

- Adds a new parking slot to the database.
- Stores the slot number with the default status as Available.

PUT /slots/:id

- Updates the status of an existing parking slot.
- Allows administrators to switch a slot between Available and Maintenance.

- If a slot is marked as Maintenance, all active bookings for that slot are automatically cancelled.

DELETE /slots/:id

- Deletes a parking slot from the system.
- Cancels all related bookings associated with the deleted slot.
- Updates booking records to maintain data consistency.

GET /create-slots

- Creates predefined demo parking slots for testing purposes.
- Inserts sample slot records into the database.

Working

1. The SlotManagement() component is created to manage all parking slot operations.
2. The component loads parking slot data from the GET /slots API when the page is opened.
3. Administrators can add new parking slots using the POST /slots API.
4. Before adding a slot, the system checks whether the slot number already exists.
5. The PUT /slots/:id API updates the status of a parking slot between Available and Maintenance.
6. If a slot is changed to Maintenance, all active bookings for that slot are cancelled automatically.
7. The DELETE /slots/:id API removes a parking slot and cancels all related bookings.
8. Dashboard summary cards are updated automatically after every operation.
9. Success messages are displayed to notify administrators of completed actions.

10. The Parking Slot Management module provides an efficient way to maintain parking slot availability and ensure proper management of parking resources.

```
app.post("/slots",async(req,res)=>{
    try{
        const slot=new Slot(req.body);
        await slot.save();

        res.json({
            success:true,
            message:"Slot Added"
        });

    }catch(err){
        res.json({
            success:false,
            message:err.message
        });
    }
});
```

```
app.put("/slots/:id", async(req,res)=>{
try{
const {status}=req.body;
const slot=await Slot.findById(req.params.id);
if(!slot){
return res.json({
message:"Slot not found"
});
}
slot.status=status;
if(status==="Maintenance"){
slot.bookedBy=null;
const users=await User.find();
for(const user of users){
let changed=false;
user.bookings.forEach(booking=>{
if(
booking.slot===slot.slotNumber &&
booking.status!=="Cancelled"
){
booking.status="Cancelled";
changed=true;
}
});
if(changed){
await user.save();
}
}
}
await slot.save();
res.json({
message:"Slot updated successfully"
});
}
catch(err){
res.status(500).json({
message:err.message
});
}
});
```

```
app.delete("/slots/:id", async(req,res)=>{
try{
const slot = await Slot.findById(req.params.id);
if(!slot){
return res.json({
message:"Slot not found"
});
}
// remove slot
await Slot.findByIdAndDelete(req.params.id);
// update users bookings
await User.updateMany(
{
"bookings.slot":slot.slotNumber
},
{
$set:{
"bookings.$[booking].status":"Cancelled"
}
},
{
arrayFilters:[
{
"booking.slot":slot.slotNumber
}
]
}
);
res.json({
message:"Slot deleted and related bookings cancelled"
});
}
catch(err){
res.status(500).json({
message:err.message
});
}
});
```

